

Your code becomes a **tree** the browser can touch.

A short walkthrough of what the frontend actually is — how a browser turns the HTML you write into a living structure called the DOM, and how that structure is what makes a page interactive.

SESSION PLAN • 60 MINUTES

0:00	Welcome — what the frontend is	5 min
0:05	How a page loads — request to render	3 min
0:08	HTML — tags, attributes, structure	16 min
0:24	The DOM — your page as a live tree	10 min
0:34	Selecting + changing the DOM — the methods	8 min
0:42	Events — reacting to the user	8 min
0:50	Build a form together — hands-on	8 min
0:58	Recap + Q&A	2 min

01 What is the frontend?

The frontend is everything a person sees and touches in the browser — the text, buttons, images, and motion. It's built from three layers, each with one job.

HTML

Structure

The content and its meaning. The skeleton of the page.

CSS

Style

How it looks — color, spacing, layout, type. The skin.

JavaScript

Behavior

What it does when you interact. The muscles and brain.

Frontend code runs in the user's browser — the part you can see. The **backend** runs on a server you can't see: databases, logic, and APIs. Today is about the frontend, and specifically the first and most foundational layer: HTML, and what the browser builds from it.

02 How a page loads

Before any of it appears, the browser runs a short pipeline. The fourth step is where today's topic is born.

1 Request

You enter a URL. The browser asks a server for the page.

2 Response

The server sends back files — HTML, CSS, and JavaScript.

3 Parse

The browser reads the HTML top to bottom and builds the **DOM** — a tree of objects representing every element.

4 Render

Using the DOM and CSS, the browser paints actual pixels on screen.

5 Run & react

JavaScript runs, changes the DOM in response to clicks and typing, and the page updates live.

03 HTML basics

HTML — *HyperText Markup Language* — describes structure using **tags** (also called **elements**). A page is just nested tags.

```
index.html

<!-- the page skeleton -->
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello world</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

Anatomy of an element

Almost every element follows the same shape: an opening tag, some content, and a closing tag. Attributes add extra information.

```
<a href="/home">Go home</a>
```

■ opening & closing tag ■ attribute (name="value") ■ content

A few elements have no content and are **self-closing**, like `<img>`, `<br>`, and `<input>`.

Attributes — extra info on a tag

Attributes configure an element: where a link points, which image to load, a name for CSS and JavaScript to grab onto. They always sit in the opening tag as `name="value"`.

```

<a href="/about" target="_blank">About</a>
<input type="email" placeholder="you@mail.com" required>
<p id="intro" class="lead">Welcome</p>
<button disabled>Sending...</button>
```

id unique name for one element	class shared name for styling / JS
src file an image or script loads	alt text shown if an image fails
href where a link goes	type kind of input or button
placeholder hint text inside a field	value the field's current value
disabled boolean — switches a control off	checked boolean — pre-ticks a box
data-* your own custom data	

`id` and `class` matter most today — they're the handles JavaScript uses to find an element in the DOM.

Tags you'll use constantly

<h1>–<h6>

Headings

<p>

Paragraph

<a>

Link

Image

** **

List & items

<div>

Block container

Inline container

<button>

Clickable button

<input>

User input field

Semantic HTML

Prefer tags that *describe their meaning* over generic `<div>` s. They make pages easier to read, better for accessibility, and friendlier to search engines.

```
<header>    <!-- top of the page -->
<nav>       <!-- navigation links -->
<main>      <!-- the main content -->
<article>   <!-- a self-contained piece -->
<footer>    <!-- bottom of the page -->
```

04 The DOM

DOM stands for **Document Object Model**. When the browser parses your HTML, it builds a tree of objects — one node per tag, per piece of text. That live tree *is* the DOM.

The HTML file is the **text you write**. The DOM is the **living tree the browser builds from it** — and the thing JavaScript can read and change on the fly.

	HTML	DOM
What it is	The text you write	A tree of live objects
Where it lives	A <code>.html</code> file on disk	The browser's memory
When it exists	Always, unchanging	Built when the page loads
Can JS change it?	No — it's just text	Yes — instantly

A useful image: HTML is the **blueprint**; the DOM is the **building** the browser constructs from it. You renovate the building, not the blueprint.

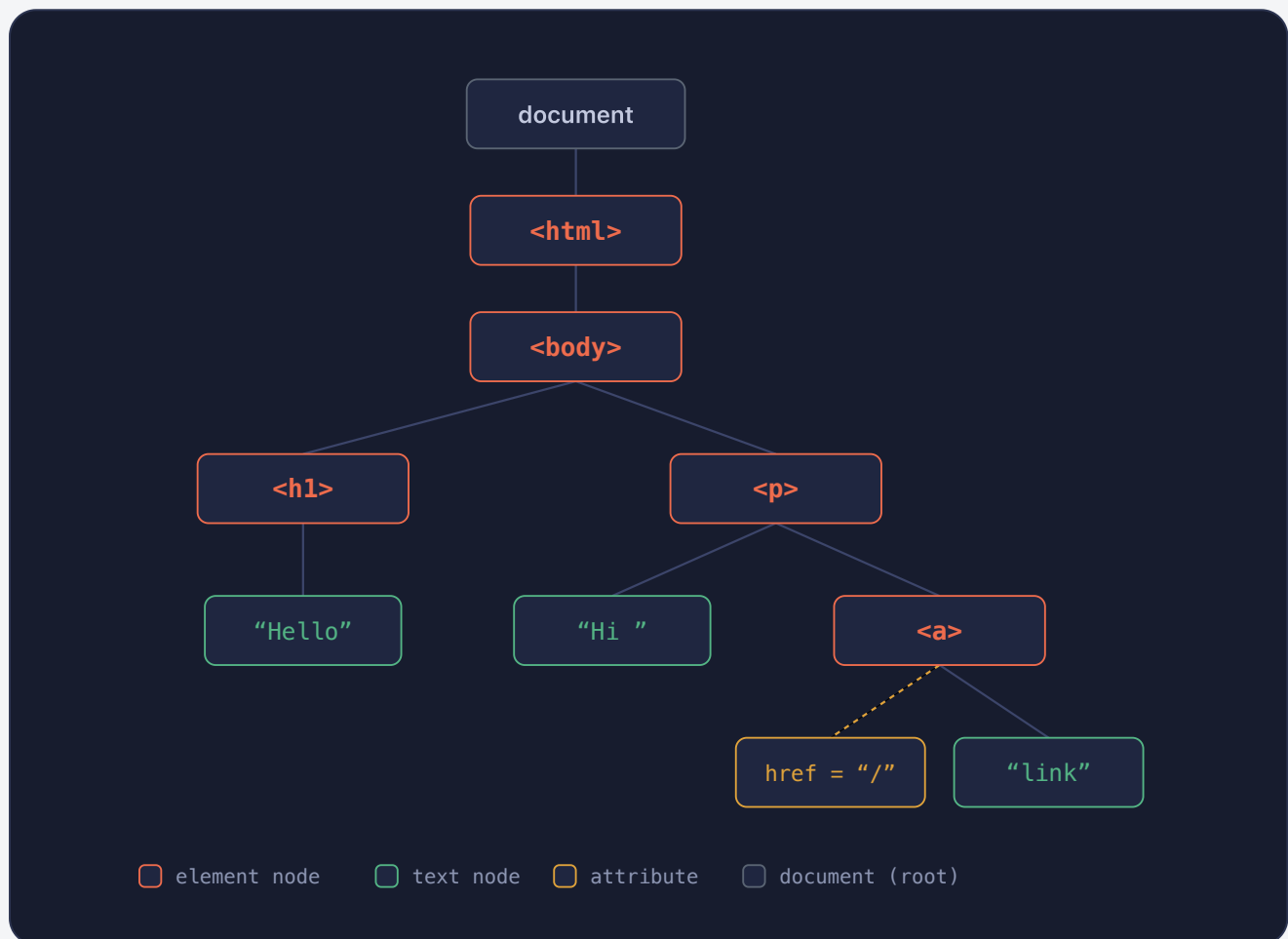
How the HTML becomes a tree

Take this small page:

page.html

```
<html>
  <body>
    <h1>Hello</h1>
    <p>Hi <a href="/">link</a></p>
  </body>
</html>
```

The browser parses it into this tree of **nodes**:



Every item is a **node**, and they relate to each other with the same words you'd use for a family tree — the exact vocabulary you'll use in code:

Parent / child

`<body>` is the parent of `<h1>` and `<p>`.

Siblings

`<h1>` and `<p>` share a parent.

Ancestor / descendant

`<a>` is a descendant of `<body>`.

Root

The `document` node at the very top.

Notice the words become their own **text nodes** — `"Hi "` and `"link"` are separate children, not part of the tags. And the `href` **attribute** hangs off its element. JavaScript can reach any of these and change them.

Now try it live: edit the HTML on the left and watch the DOM tree rebuild on the right. This is exactly what the browser does as it reads your file.

HTML → DOM TREE

LIVE

YOU WRITE

```
<section>
  <h1 class="title">Frontend</h1>
  <p>Hello <a href="#">link</a></p>
```

BROWSER BUILDS

```
<section>
  <h1> class="title"
    text NODE "Frontend"
  <p>
    text NODE "Hello"
    <a> href="#"
      text NODE "link"
```

Every box is a **node**. Tags become element nodes (orange), their attributes hang off them (amber), and the words inside become text nodes (grey). Nodes have parents, children, and siblings — just like a family tree.

05 How the page gets painted

The DOM by itself is invisible — it's just objects in memory. Turning it into pixels takes a short pipeline the browser runs on every frame.

First, meet the CSSOM

Just as the browser turns your HTML into the DOM, it turns your CSS into the **CSSOM** (CSS Object Model) — a tree of style rules in memory. Neither tree can draw the page alone; the browser combines them.

	DOM	CSSOM
Built from	your HTML	your CSS
Represents	structure — what's there	styles — how it looks
A node is	an element or text	a rule + its properties
JS reaches it via	<code>document</code>	<code>document.styleSheets</code>

Writing `el.style.color = "red"` edits the CSSOM, exactly like `el.textContent = "Hi"` edits the DOM. The framing: **DOM = what's on the page**, **CSSOM = what it looks like**, **render tree = the two zipped together**.

The render pipeline

The browser parses both files into trees, merges them into a **render tree** of just the visible nodes, then lays them out, paints the pixels, and composites the layers to the screen:



"Printed" really means the **paint + composite** steps. Nodes in the DOM but not visible — `<head>`, anything with `display:none` — never reach the render tree, so they're never painted.

What happens when JavaScript updates content

When you run `el.textContent = "Hi"`, the screen doesn't change instantly. The browser edits the DOM object, marks that part *dirty*, and **batches** the work. On the next frame (~every 16 ms, 60 times a second) it re-runs *only the steps your change touched* — that's the dashed loop above.

The key insight: not all changes cost the same.

YOU CHANGE...	BROWSER RE-RUNS	COST
Add/remove a node, width, font, position	Layout → Paint → Composite	expensive
Colour, background, shadow	Paint → Composite	medium
<code>transform</code> , <code>opacity</code>	Composite only	cheap (GPU)

That's why smooth animations use `transform` and `opacity` — they skip layout and paint entirely. And it's exactly what the counter and form demos below do: change one DOM node, the browser repaints it on the next frame, and it feels instant.

06 Making it interactive

JavaScript reaches into the DOM to **find** elements, **change** them, and **listen** for what the user does. Three lines cover most of it:

```
script.js

// 1. find an element in the DOM
const btn = document.querySelector("#like");

// 2. listen for an event
btn.addEventListener("click", () => {
  // 3. change the DOM
  btn.textContent = "Liked!";
});
```

The DOM verbs you'll actually use

A small set of methods covers almost everything — find a node, read or change it, add or remove one, and listen for events.

<code>querySelector()</code> find the first match	<code>querySelectorAll()</code> find every match
<code>getElementById()</code> find one element by id	<code>.textContent</code> read / set the text

.innerHTML read / set inner HTML	.setAttribute() set an attribute
.classList.toggle() add / remove a class	.style.color change one style
createElement() make a new node	.appendChild() add a node to the tree
.remove() delete a node	addEventListener() listen for an event

A few more things you can do

```
examples.js

// change text and style
title.textContent = "Updated!";
title.style.color = "tomato";

// toggle a CSS class on/off
box.classList.toggle("active");

// build a new element and add it to the page
const li = document.createElement("li");
li.textContent = "New item";
list.appendChild(li);

// read what a user typed
const name = input.value;
```

Events — reacting to the user

An **event** is something that happens: a click, a keypress, a form submit. You attach a function with `addEventListener` and the browser runs it each time the event fires.

click element is clicked or tapped	input value changes as you type
change value committed (blur / pick)	submit a form is submitted
keydown a key is pressed	mouseover pointer enters an element
load page or image finished loading	DOMContentLoaded the DOM is ready to use

The two demos below are real — they're running this exact pattern right now on this page. Click and type.

EVENT → CHANGE THE DOM

Clicked 0 times

Each click runs **count++** and rewrites the button's `textContent`.

INPUT → LIVE UPDATE

Type your name

Hello there 🖐️

Typing fires an **input** event that updates the heading instantly.

Putting it together — a tiny form

This combines everything: HTML structure, a `<label>` tied to each field, a `submit` event, reading values, and writing the result back into the DOM. It's live — fill it in and submit.

Your name

Ada Lovelace

Favourite tag

`<section>`



Submit

Your result will appear here.

form.js

```
const btn = document.querySelector("#su-go");

btn.addEventListener("click", () => {
  // read the values the user entered
  const name = document.querySelector("#su-name").value;
  const tag = document.querySelector("#su-role").value;
  // write the result back into the DOM
  document.querySelector("#su-out").textContent =
    name + " likes the " + tag + " tag.";
});
```

07 The mental model

summary

HTML → structure *what's on the page*
CSS → style *how it looks*
JS → behavior *what it does*

DOM → the live tree the browser builds from HTML,
and the thing JavaScript changes to make
the page come alive.

08 Glossary

Element / tag	A building block of HTML, like <code><p></code> .
Attribute	Extra info on a tag — <code>href</code> , <code>class</code> , <code>id</code> .
Node	Any single item in the DOM tree — an element or a piece of text.
Parent / child	How nodes nest. A tag inside another is its child.
Event	Something that happens — a click, a keypress, the page loading.
CSSOM	The CSS turned into a tree of style objects — the DOM's styling counterpart.
Render tree	DOM + CSSOM merged into just the visible nodes and their styles.
Reflow / repaint	Recomputing layout / redrawing pixels after something changes.
Render	The browser drawing the page on screen from the DOM.
Semantic HTML	Tags that describe meaning, like <code><nav></code> .
Self-closing	A tag with no content, like <code></code> .

09 Full tag reference

Every HTML element, grouped by what it's for. You won't use most of these daily — treat this as a lookup, not a memorisation list. The handful in each group marked in the session are the ones worth knowing cold.

A Document & metadata

The page's frame and the information about it — most of this lives in `<head>` and never shows on screen.

<code><!DOCTYPE></code> declares HTML5 (standards mode)	<code><html></code> root of the whole page
<code><head></code> metadata container	<code><title></code> page title (tab & search)
<code><meta></code> charset, viewport, SEO	<code><link></code> external resource (CSS, icon)
<code><style></code> embedded CSS	<code><base></code> base URL for all links
<code><body></code> everything visible	

B Sectioning & layout

Semantic landmarks that describe the regions of a page.

<code><header></code> top / introductory area	<code><nav></code> navigation links
<code><main></code> primary content	<code><section></code> thematic group
<code><article></code> self-contained piece	<code><aside></code> sidebar / tangent
<code><footer></code> bottom area	<code><h1>–<h6></code> headings, big to small
<code><hgroup></code> heading + subheading	<code><address></code> contact information

C Text blocks

Block-level content — each starts on its own line.

<code><p></code> paragraph	<code><hr></code> thematic break (rule)
<code><pre></code> preformatted text	<code><blockquote></code> quoted block
<code></code> unordered list	<code></code> ordered list
<code></code> list item	<code><dl></code> description list
<code><dt></code> description term	<code><dd></code> description detail
<code><figure></code> self-contained media	<code><figcaption></code> caption for a figure
<code><div></code> generic block container	<code><menu></code> list of commands

D Inline text

Phrasing content that flows inside a line of text.

<a> link (anchor)	 generic inline container
 important (bold)	 emphasis (italic)
 stylistic bold	<i> stylistic italic
<u> underline	<s> strikethrough
<small> fine print	<mark> highlighted text
<abbr> abbreviation	<cite> title of a work
<q> short inline quote	<code> code snippet
<kbd> keyboard input	<samp> sample output
<var> variable	<sub> subscript
<sup> superscript	<time> date / time
<data> machine-readable value	
 line break
<wbr> line-break opportunity	<dfn> definition term
<bdi> bidirectional isolate	<bdo> bidirectional override
<ruby> annotation wrapper	<rt> annotation text
<rp> annotation fallback	

E Images & media

Pictures, audio, and video.

 image	<picture> responsive image wrapper
<source> media source option	<audio> sound
<video> video	<track> captions / subtitles
<map> image map	<area> clickable map region

F Embedded content

Pulling in other documents, apps, or graphics.

<iframe> embedded web page	<embed> external app / plugin
<object> external resource	<param> parameter for object
<canvas> drawing surface (JS)	<svg> vector graphics
<math> math notation	

G Tables

Tabular data in rows and columns.

<table> the table	<caption> table title
<thead> header rows	<tbody> body rows
<tfoot> footer rows	<tr> table row
<th> header cell	<td> data cell
<colgroup> column group	<col> column properties

H Forms & input

Collecting input from the user.

<form> input form	<input> single-line field
<textarea> multi-line field	<button> button
<select> dropdown	<option> dropdown choice
<optgroup> group of options	<label> label for a field
<fieldset> group of fields	<legend> fieldset caption
<datalist> autocomplete options	<output> calculation result
<progress> progress bar	<meter> gauge / measurement

I Interactive & scripting

Built-in interactivity and where JavaScript lives.

<details> expandable disclosure	<summary> heading for details
<dialog> dialog / modal	<script> JavaScript
<noscript> fallback if JS is off	<template> inert reusable markup
<slot> web-component placeholder	

Void (self-closing) elements. These have no content and no closing tag: `<area>` `<base>` `<br>` `<col>` `<embed>` `<hr>` `<img>` `<input>` `<link>` `<meta>` `<source>` `<track>` `<wbr>`.

× Deprecated — don't use

Still recognised by browsers, but removed from the standard. Use CSS instead.

<center> → use CSS text-align	 → use CSS color / font
<big> → use CSS font-size	<strike> → use <code><s></code> or <code></code>
<tt> → use <code><code></code>	<marquee> → use CSS animation
<acronym> → use <code><abbr></code>	<frame> → use <code><iframe></code>
<frameset> → obsolete	